# 代码片段学习文档

这个文档只记录你明确指定过的代码内容，不主动挑选片段。

## 使用规则

- 只有当你明确指定某个文件、函数、类、模块、调用链或代码片段时，才进行讲解和记录。
- 先在对话里用中文讲解清楚，不要立即写入本文件。
- 只有在你明确同意“加入学习文档”之后，才把这次讲解整理后写入本文件。
- 如果你没有明确同意，就只保留对话讲解，不写入文档。
- 写入时不大段粘贴源码，优先记录理解和关系。
- 顺序编号按写入本文件的先后递增。
- 记录时间使用实际写入时的时间。

## 记录模板

### 记录 N

- 顺序编号: N
- 记录时间: YYYY-MM-DD HH:mm:ss (Asia/Shanghai)
- 标题:
- 文件路径:
- 相关函数/类:
- 核心解释:
- 调用关系:
- 后续阅读建议:
- 我的学习状态:

## 说明

- 这里的重点是“怎么读懂代码”，不是把源码抄进来。
- 如果后续你指定了一个具体目标，我会先在对话中讲解，再等你确认是否加入这里。

## 学习记录

### 记录 1

- 顺序编号: 1
- 记录时间: 2026-03-22 00:10:03 (Asia/Shanghai)
- 标题: 核心 agent 实现：`AgentLoop` 如何把消息变成回复
- 文件路径:
  - `nanobot/agent/loop.py`

  - `nanobot/agent/context.py`
  - `nanobot/agent/subagent.py`
  - `nanobot/agent/tools/message.py`
  - `nanobot/agent/tools/spawn.py`
  - `nanobot/bus/queue.py`
  - `nanobot/bus/events.py`
  - `nanobot/session/manager.py`
  - `nanobot/providers/base.py`
- 相关函数/类:
  - `AgentLoop`
  - `AgentLoop.run()`
  - `AgentLoop._process_message()`
  - `AgentLoop._run_agent_loop()`
  - `ContextBuilder.build_system_prompt()`
  - `ContextBuilder.build_messages()`
  - `SubagentManager.spawn()`
  - `SubagentManager._announce_result()`
  - `MessageTool.execute()`
  - `SpawnTool.execute()`
  - `MessageBus`
  - `InboundMessage`
  - `OutboundMessage`
  - `SessionManager.get_or_create()`
  - `Session.get_history()`
  - `LLMProvider.chat_with_retry()`
- 代码片段:

1. `AgentLoop.__init__()` 先把核心部件装配起来，这一步决定整个 agent 运行时会有哪些能力。

```
self.context = ContextBuilder(workspace)
self.sessions = session_manager or SessionManager(workspace)
self.tools = ToolRegistry()
self.subagents = SubagentManager(
    provider=provider,
    workspace=workspace,
    bus=bus,
    model=self.model,
)
self.memory_consolidator = MemoryConsolidator(
    workspace=workspace,
    provider=provider,
    model=self.model,
    sessions=self.sessions,
    context_window_tokens=context_window_tokens,
    build_messages=self.context.build_messages,
```

```
    get_tool_definitions=self.tools.get_definitions,
)
self._register_default_tools()
```

解释：这一段处在 `AgentLoop` 的初始化阶段，作用不是处理消息，而是把后面会用到的上下文、会话、工具、子 agent、记忆整理器先准备好。`ContextBuilder` 决定 prompt 怎么拼，`SessionManager` 决定会话怎么存，`ToolRegistry` 决定模型能调用哪些工具，`SubagentManager` 决定后台任务怎么跑，`MemoryConsolidator` 决定什么时候把旧对话压缩进长期记忆。最后 `self._register_default_tools()` 把默认能力挂到工具注册表里，这样后续 `_run_agent_loop()` 才能直接调用。

2. `AgentLoop._process_message()` 的 system 分支会把子 agent 的回流结果重新送回主流程，这就是主 agent 和子 agent 接起来的关键点。

```
if msg.channel == "system":
    channel, chat_id = (msg.chat_id.split(":", 1) if ":" in msg.chat_id
                        else ("cli", msg.chat_id))
    session = self.sessions.get_or_create(key)
    current_role = "assistant" if msg.sender_id == "subagent" else "user"
    messages = self.context.build_messages(
        history=history,
        current_message=msg.content, channel=channel, chat_id=chat_id,
        current_role=current_role,
    )
    final_content, _, all_msgs = await self._run_agent_loop(messages)
    return OutboundMessage(channel=channel, chat_id=chat_id,
                           content=final_content or "Background task completed.")
```

解释：这里的重点是 `msg.sender_id == "subagent"` 这条判断。它决定这条 system 消息应该被当成 `assistant` 还是 `user`。如果是子 agent 回流，主 agent 就把它当成助手结果继续总结；如果不是子 agent，就按普通系统消息处理。最后还是交给 `_run_agent_loop()`，说明 system 消息并不是终点，而是重新进入主 agent 推理链路的入口。

3. `AgentLoop._process_message()` 的普通消息分支先整理会话和工具上下文，再进入模型循环。

```
await self.memory_consolidator.maybe_consolidate_by_tokens(session)
self._set_tool_context(msg.channel, msg.chat_id, msg.metadata.get("message_id"))
if message_tool := self.tools.get("message"):
    if isinstance(message_tool, MessageTool):
        message_tool.start_turn()

history = session.get_history(max_messages=0)
initial_messages = self.context.build_messages(
    history=history,
    current_message=msg.content,
    media=msg.media if msg.media else None,
    channel=msg.channel, chat_id=msg.chat_id,
)
final_content, _, all_msgs = await self._run_agent_loop(
    initial_messages, on_progress=on_progress or _bus_progress,
)
```

解释：这段是普通消息的主干前半段。先做一次记忆整理，避免上下文太长；再把 channel/chat_id/message_id 传给需要路由信息的工具；然后重置 `message` 工具的轮次状态，防止上一轮的发送标记污染这一轮。接着从 session 取历史，由 `ContextBuilder` 拼出

`initial_messages`，最后才进入 `_run_agent_loop()`。这说明真正的智能推理并不是直接从用户输入开始，而是从“历史 + prompt + 工具上下文”一起开始。

4. `ContextBuilder.build_system_prompt()` 决定模型到底看见什么，这一步把仓库文档和记忆带进 prompt。

```
BOOTSTRAP_FILES = ["AGENTS.md", "SOUL.md", "USER.md", "TOOLS.md"]

bootstrap = self._load_bootstrap_files()
if bootstrap:
    parts.append(bootstrap)

memory = self.memory.get_memory_context()
if memory:
    parts.append(f"# Memory\n\n{memory}")

always_skills = self.skills.get_always_skills()
if always_skills:
    always_content = self.skills.load_skills_for_context(always_skills)
    if always_content:
        parts.append(f"# Active Skills\n\n{always_content}")
```

解释：这里的关键不是“拼字符串”本身，而是把不同来源的上下文分层放进 system prompt。`BOOTSTRAP_FILES` 会读取 workspace 根目录里的行为文档，`memory` 会带上长期记忆，`skills_summary` 会告诉模型有哪些技能可用。也就是说，agent 的行为不是写死在代码里，而是通过这些文档和记忆层控制出来的。

5. `SubagentManager._announce_result()` 会把子 agent 的结果包装成 system 消息，重新送回主 agent。

```
msg = InboundMessage(
    channel="system",
    sender_id="subagent",
    chat_id=f"{origin['channel']}:{origin['chat_id']}",
    content=announce_content,
)

await self.bus.publish_inbound(msg)
```

解释：这两步是子 agent 回流的核心。`channel="system"` 表示这不是普通用户消息，`sender_id="subagent"` 让主 agent 能识别它来自后台子任务，`chat_id` 则把它重新绑定回原来的会话。最后 `publish_inbound()` 把它投回总线，所以主 agent 后面会像处理其他系统消息一样处理它。

- 核心解释:
  - 这个项目的核心不是“一个直接回答问题的模型对象”，而是 `AgentLoop` 这个调度中心。它把入口消息、会话历史、长期记忆、技能、工具、子 agent 和模型调用串成一条完整链路。
  - `MessageBus` 负责把渠道层和核心逻辑解耦：入口把消息放进 `inbound`，`AgentLoop` 消费后，再把回复或进度消息放进 `outbound`。
  - `AgentLoop.__init__()` 会先装配 `ContextBuilder`、`SessionManager`、`ToolRegistry`、`SubagentManager`、`MemoryConsolidator`，再注册默认工具，所以它更像“总指挥”，不是单一模型封装。

  - `AgentLoop.run()` 负责从 `MessageBus.inbound` 取消息，先拦截 `/stop` 和 `/restart`，再把普通消息交给 `_dispatch()` 和 `_process_message()`。
  - `_process_message()` 是单条消息的主干：先分 `system` 和普通消息两条路，再处理会话、记忆、工具上下文、prompt 拼装和最终输出。
  - `system` 分支主要给子 agent 回流结果使用。它会把 `sender_id == "subagent"` 的内容当成 `assistant` 角色重新进入主流程，让主 agent 继续总结，而不是直接把内部结果暴露给用户。
  - 普通消息分支会先处理 `/new`、`/help` 等命令，再调用记忆整理，设置工具上下文，重置 `MessageTool` 的轮次状态，取会话历史，拼出 `initial_messages`，最后进入 `_run_agent_loop()`。
  - `_run_agent_loop()` 才是真正的“模型-工具迭代器”：先 `chat_with_retry()`，如果模型产生 tool call，就执行工具并把工具结果写回消息列表；如果模型直接回答，就清理 内容并结束。
  - `ContextBuilder.build_system_prompt()` 决定模型看见什么。它会读取 workspace 根目录里的 `AGENTS.md`、`SOUL.md`、`USER.md`、`TOOLS.md`，再叠加长期记忆和技能摘要，因此仓库里的文档会直接影响 agent 行为。
  - `ContextBuilder.build_messages()` 会把 runtime metadata 和当前用户消息合并成一个 user message，避免某些 provider 对连续同角色消息不兼容。
  - `SubagentManager.spawn()` 会在后台开一个子 agent；子 agent 完成后通过 `_announce_result()` 把结果包装成 `channel="system"` 的 `InboundMessage` 回到总线，主 agent 再把它当作系统回流消息处理。
  - `MessageTool` 的 `_sent_in_turn` 用来避免一轮里重复发两次回复；如果模型已经用工具主动发过消息，`_process_message()` 就不会再返回一条最终 OutboundMessage。
  - 这套结构的设计目标是解耦和可理解性：prompt、记忆、会话、工具、子 agent、模型 provider 都各自有清晰边界，便于学习和维护。
- 调用关系:
  - `CLI / gateway -> MessageBus.inbound -> AgentLoop.run() -> AgentLoop._process_message() -> AgentLoop._run_agent_loop() -> ToolRegistry / SubagentManager / MemoryConsolidator -> OutboundMessage -> MessageBus.outbound`
  - `SubagentManager._announce_result() -> MessageBus.inbound -> AgentLoop._process_message(system) -> AgentLoop._run_agent_loop() -> OutboundMessage`
- 后续阅读建议:
  - 先读 `nanobot/agent/context.py`，重点看 `build_system_prompt()` 和 `build_messages()`，理解模型输入是怎么拼出来的。
  - 再读 `nanobot/session/manager.py`，理解历史消息是怎么保存、截断和对齐 tool call 边界的。
  - 接着读 `nanobot/agent/memory.py`，理解长期记忆是怎么整理进 `MEMORY.md` 和 `HISTORY.md` 的。
  - 然后读 `nanobot/agent/subagent.py` 和 `nanobot/agent/tools/spawn.py`，理解子 agent 如何启动并回流。
  - 最后读 `nanobot/providers/base.py`，理解模型调用、重试和 provider 抽象层的职责。
- 我的学习状态:

- 已掌握主干思路，知道 `AgentLoop` 是核心调度中心。
- 已能区分 `session`、`memory`、`prompt`、`tool`、`subagent` 和 `provider` 的职责。
- 下一步准备继续细读 `ContextBuilder.build_system_prompt()`。

## 记录 2

- 顺序编号: 2
- 记录时间: 2026-03-25 10:55:08 (Asia/Shanghai)
- 标题: `openai_codex_provider.py` 入门：文件作用与“类继承 + super()”
- 文件路径:
  - `nanobot/providers/openai_codex_provider.py`
  - `nanobot/providers/base.py`
- 相关函数/类:
  - `OpenAICodexProvider`
  - `OpenAICodexProvider.__init__()`
  - `LLMProvider`
  - `LLMProvider.__init__()`
  - `super()`
- 代码片段:

1. `OpenAICodexProvider` 通过继承 `LLMProvider`，复用 provider 的通用初始化逻辑，再补自己的默认模型配置。

   ```
   class OpenAICodexProvider(LLMProvider):
       def __init__(self, default_model: str = "openai-codex/gpt-5.1-codex"):
           super().__init__(api_key=None, api_base=None)
           self.default_model = default_model
   ```

   解释：这一段在 `openai_codex_provider.py` 的类定义开头，是整个 Codex provider 的初始化入口。`class OpenAICodexProvider(LLMProvider)` 表示它继承父类 `LLMProvider`，说明它不是从零开始实现一个 provider，而是站在统一抽象接口上扩展。`super().__init__(api_key=None, api_base=None)` 会先执行父类初始化，把所有 provider 共有的状态先准备好；这里传 `None`，表示这个 provider 不走普通的 `api_key/api_base` 配置，而是走 Codex OAuth。最后 `self.default_model = default_model` 再补上这个子类自己的特有属性。

2. 父类 `LLMProvider` 负责 provider 通用状态，子类只需要复用它，不需要重复写。

   ```
   def __init__(self, api_key: str | None = None, api_base: str | None = None):
       self.api_key = api_key
       self.api_base = api_base
       self.generation = GenerationSettings()
   ```

   解释：这一段在 `base.py` 里，是所有 provider 的公共初始化逻辑。它统一初始化了 `api_key`、`api_base` 和 `generation`。因此 `OpenAICodexProvider` 调用 `super().__init__()` 之后，就自动拥有这些公共属性，不需要再写一遍。项目这里的设计思路很清楚：父类负责“所有 provider 共通的部分”，子类负责“Codex 这家独有的部分”。

- 核心解释:

  - 这个文件的作用是把 `nanobot` 内部统一的消息格式，转换成 OpenAI Codex Responses API 能接受的格式，再用异步 HTTP 流式请求把结果拿回来，最后解析成项目内部统一的 `LLMResponse` 对象。
  - 从学习角度看，`openai_codex_provider.py` 是一个很典型的“适配层”文件：上游接项目自己的 provider 抽象，下游接外部 Codex API。
  - 这次先聚焦的 Python 知识点是“类继承与 `super()`”。它的核心不是语法本身，而是对象初始化职责如何在父类和子类之间分层。
  - 在这个文件里，`LLMProvider` 负责共通字段，`OpenAICodexProvider` 负责 Codex 专有配置，所以要先调用 `super().__init__()`，再写 `self.default_model`。
- 调用关系:
  - `OpenAICodexProvider(LLMProvider) -> OpenAICodexProvider.__init__() -> super().__init__() -> LLMProvider.__init__()`
  - `LLMProvider.__init__()` 初始化通用 provider 状态 -> `OpenAICodexProvider.__init__()` 补充 `default_model`
- 后续阅读建议:
  - 下一步先继续学 `__init__` 和 `self`，这样会更容易理解对象初始化。
  - 然后读 `openai_codex_provider.py` 里的类型注解，例如 `str | None`、`list[dict[str, Any]]`。
  - 再往后看 `async def` / `await`，因为这个文件的主流程大量依赖异步网络请求。
- 我的学习状态:
  - 已知道 `openai_codex_provider.py` 在项目里属于 provider 适配层。
  - 已理解 `OpenAICodexProvider` 为什么继承 `LLMProvider`，以及 `super().__init__()` 的作用。
  - 下一步准备继续学习这个文件里的 `__init__`、`self` 或类型注解。

## 记录 3

- 顺序编号: 3
- 记录时间: 2026-03-30 16:30:00 (Asia/Shanghai)
- 标题: Slash 命令系统：`CommandRouter` 如何路由和分发对话中的命令
- 文件路径:
  - `nanobot/command/router.py`
  - `nanobot/command/builtin.py`
  - `nanobot/agent/loop.py`
- 相关函数/类:
  - `CommandRouter`
  - `CommandRouter.priority()`
  - `CommandRouter.exact()`
  - `CommandRouter.prefix()`
  - `CommandRouter.dispatch_priority()`
  - `CommandRouter.dispatch()`

- `CommandContext`
- `register_builtin_commands()`
- `cmd_stop()`
- `cmd_restart()`
- `cmd_status()`
- `cmd_new()`
- `cmd_help()`

- 代码片段:

1. `CommandRouter` 定义了三层命令匹配机制，优先级从高到低：priority > exact > prefix > interceptors。

```
class CommandRouter:
    def __init__(self) -> None:
        self._priority: dict[str, Handler] = {}    # 优先命令（在锁之前处理）
        self._exact: dict[str, Handler] = {}        # 精确匹配
        self._prefix: list[tuple[str, Handler]] = []  # 前缀匹配
        self._interceptors: list[Handler] = []       # 拦截器（兜底）
```

解释：`CommandRouter` 是一个纯字典驱动的命令分发器。它不处理业务逻辑，只负责把命令文本映射到对应的处理函数。`_priority` 存放需要立即响应的命令（如 `/stop`），这些命令在获取分布式锁之前就会执行，避免等待；`_exact` 存放精确匹配命令；`_prefix` 存放前缀匹配（如 `/team xxx`）；`_interceptors` 是最后的兜底拦截器。

2. `CommandContext` 是命令处理函数的统一上下文，所有命令处理器都接收这个参数。

```
@dataclass
class CommandContext:
    msg: InboundMessage      # 原始入站消息
    session: Session | None  # 当前会话
    key: str                 # 会话标识
    raw: str                 # 原始命令文本
    args: str = ""           # 命令参数（前缀匹配时提取）
    loop: Any = None         # AgentLoop 引用
```

解释：这个 dataclass 把命令处理需要的所有信息打包在一起。`msg` 是原始消息；`session` 是当前会话对象；`key` 是会话的唯一标识；`raw` 是用户输入的原始命令文本；`args` 在前缀匹配时会被自动填充为命令后的参数部分；`loop` 是 AgentLoop 的引用，让命令处理器能访问取消任务、重启等能力。

3. `register_builtin_commands()` 把内置命令注册到路由器，区分优先级和精确匹配。

```
def register_builtin_commands(router: CommandRouter) -> None:
    router.priority("/stop", cmd_stop)      # 优先级：需要立即响应
    router.priority("/restart", cmd_restart)
    router.priority("/status", cmd_status)
    router.exact("/new", cmd_new)           # 精确匹配
    router.exact("/help", cmd_help)
```

解释：这一步在 `AgentLoop.__init__()` 里被调用，把 5 个内置命令挂到路由器上。`/stop`、`/restart`、`/status` 用 `priority()` 注册，因为它们需要在锁之前执行，避免被阻塞；`/new`、`/help` 用 `exact()` 注册，走普通的精确匹配流程。

4. `cmd_stop()` 是典型的优先命令实现，直接取消任务不等待锁。

```
async def cmd_stop(ctx: CommandContext) -> OutboundMessage:
    loop = ctx.loop
    msg = ctx.msg
    tasks = loop._active_tasks.pop(msg.session_key, [])
    cancelled = sum(1 for t in tasks if not t.done() and t.cancel())
    for t in tasks:
        try:
            await t
        except (asyncio.CancelledError, Exception):
            pass
    sub_cancelled = await loop.subagents.cancel_by_session(msg.session_key)
    total = cancelled + sub_cancelled
    content = f"Stopped {total} task(s)." if total else "No active task to stop."
    return OutboundMessage(channel=msg.channel, chat_id=msg.chat_id, content=content)
```

   解释：这个函数展示了优先命令的典型实现模式。它从 `ctx.loop._active_tasks` 取出当前会话的所有活跃任务，逐个取消；然后通过 `subagents.cancel_by_session()` 取消子代理。最后返回一个 `OutboundMessage`，格式和其他消息输出一致。重点是它不等待任何锁，直接执行，这样用户按 `/stop` 时能立即得到响应。

5. `CommandRouter.dispatch_priority()` 和 `dispatch()` 的分发逻辑。

```
async def dispatch_priority(self, ctx: CommandContext) -> OutboundMessage | None:
    handler = self._priority.get(ctx.raw.lower())
    if handler:
        return await handler(ctx)
    return None

async def dispatch(self, ctx: CommandContext) -> OutboundMessage | None:
    cmd = ctx.raw.lower()
    if handler := self._exact.get(cmd):
        return await handler(ctx)
    for pfx, handler in self._prefix:
        if cmd.startswith(pfx):
            ctx.args = ctx.raw[len(pfx):]
            return await handler(ctx)
    for interceptor in self._interceptors:
        result = await interceptor(ctx)
        if result is not None:
            return result
    return None
```

   解释：`dispatch_priority()` 只检查 `_priority` 字典，用于在锁之前快速响应。`dispatch()` 则是完整的分发流程：先查 `_exact`，再按最长前缀匹配 `_prefix`（列表已按长度降序排列），最后尝试 `_interceptors`。前缀匹配时会自动提取 `args`。如果都没有匹配，返回 `None`，表示这不是一个命令。

- 核心解释:
  - Slash 命令系统的核心是"路由 + 处理器"的分离设计。`CommandRouter` 只负责路由，不关心命令具体做什么；各个 `cmd_*` 函数只负责处理，不关心命令怎么被分发。
  - 优先级设计是为了处理 `/stop` 这类需要立即响应的命令。它们在 AgentLoop 获取分布式锁之前就会执行，避免用户等待。

  - `CommandContext` 把命令处理需要的所有上下文打包成一个对象，避免函数签名过长，也方便后续扩展。
  - 命令处理器统一返回 `OutboundMessage | None`，这样命令结果可以复用消息发送流程，不需要单独的输出机制。
  - 整个系统是可扩展的：外部模块可以调用 `router.priority()` / `router.exact()` / `router.prefix()` 注册自己的命令，不需要修改 builtin.py。
- 调用关系:
  - `AgentLoop.__init__() -> register_builtin_commands(self.commands) -> router.priority() / router.exact()`
  - `AgentLoop.run() -> CommandRouter.is_priority() -> CommandRouter.dispatch_priority() -> cmd_stop / cmd_restart / cmd_status`
  - `AgentLoop.run() -> CommandRouter.dispatch() -> cmd_new / cmd_help / prefix handlers / interceptors`
- 后续阅读建议:
  - 先读 `nanobot/agent/loop.py` 的 `run()` 方法，看 `is_priority()` 和 `dispatch()` 是怎么被调用的。
  - 再看 `nanobot/bus/events.py` 理解 `InboundMessage` 和 `OutboundMessage` 的完整结构。
  - 如果想了解如何扩展命令系统，可以看是否有其他模块调用 `router.prefix()` 或 `router.intercept()`。
- 我的学习状态:
  - 已理解 slash 命令的三层路由机制：priority > exact > prefix > interceptors。
  - 已知道为什么 `/stop` 用 `priority()` 注册而不是 `exact()`。
  - 下一步准备看 `AgentLoop.run()` 中命令分发的实际调用点。